2026 C4 初赛文档

KubeWise：面向 Kubernetes 集群的智能自动运维 Agent 系统

目录

1. 项目说明
2. 问题分析与需求背景
3. 竞品分析
4. 实现方案
5. 技术栈
6. 预期效果
7. 参考文献

一、项目说明

1.1 项目概述

KubeWise 是一个面向 Kubernetes 集群的智能自动运维 Agent 系统，旨在将大语言模型（LLM）的自然语言理解与推理能力，与 Kubernetes 丰富的 API 生态深度融合，实现**一句话操作**、**智能查询**、**自动故障排除**与**安全合规检测**四大核心功能。运维人员无需记忆复杂的 kubectl 命令或深入了解 Kubernetes 底层 API，只需用自然语言描述意图，KubeWise 便可自主规划、调用工具并完成任务。

本系统定位于解决云原生基础设施运维的**知识门槛高、操作链路长、故障响应慢、安全审计难**四大核心痛点，适用于运维工程师、开发团队以及高校实验室场景，同时具备较强的产业落地潜力。

1.2 项目选题与赛道契合度

根据 2026 年 C4-网络技术挑战赛竞赛规程，A 系列作品须关联以下网络技术分支之一：**云计算、网络智能、软件定义网络**等 。本项目聚焦 Kubernetes 作为云计算基础设施核心编排引擎的自动化运维场景，融合了**云计算**（Kubernetes 集群管理）、**网络**

智能（LLM 驱动的智能决策）与**网络安全**（RBAC 权限审计）三大方向，高度契合 A 赛道的选题要求。

在创新维度上，本项目并非对已有工具的简单集成，而是构建了一个**多 Agent 协作、有状态推理的运维智脑**，在技术方法和应用场景两个层面均具有显著的开拓性。

二、问题分析与需求背景

2.1 Kubernetes 运维的复杂性挑战

Kubernetes 已成为现代云原生基础设施的事实标准，根据 CNCF 2024 年度调查，**全球超过 84% 的受访组织在生产环境中使用 Kubernetes**，容器化工作负载占所有已部署应用的 55% 以上，Kubernetes 在企业中的采用率已超过 70%。然而，Kubernetes 的强大功能背后隐藏着极高的运维复杂度：

当集群中的某个工作负载出现异常时，工程师往往需要在 `kubectl describe`、容器日志、Prometheus 指标、Helm 配置值、历史事件记录以及内部 Runbook 之间来回切换，综合判断故障根因 。这一过程不仅耗时，还高度依赖工程师的专项经验积累，导致**知识碎片化**成为制约运维效率的核心瓶颈。

2.2 具体场景中的核心痛点

通过对典型运维工作流程的分析，可归纳出四类高频、高价值的痛点场景：

（1）操作门槛高：部署一个 ArgoCD、Cert-Manager 等常用组件，往往需要工程师熟悉 Helm Chart、Namespace 规划、RBAC 配置等多个层次的知识；即便是简单的扩缩容操作，也需要掌握正确的 kubectl patch 或 scale 命令格式。对于初级运维或开发同学，这是明显的效率瓶颈。

（2）跨资源查询复杂：Kubernetes 中的资源高度关联，例如"哪个 PV 占用空间最大，它绑定的 PVC 挂载到了哪个 Pod" 这类问题，需要跨 PV、PVC、Pod 三类资源进行联合查询与推理，单一 kubectl 命令无法直接回答，通常需要多条命令的结果人工拼接分析。

（3）故障响应周期长：容器崩溃或健康检查失败时，从发现问题到定位根因，平均需要工程师花费数十分钟甚至更长时间，拉取日志、分析事件流、排查依赖链路。故障的平均恢复时间（MTTR）居高不下，直接影响服务可用性。

（4）安全合规审计缺失：随着集群规模的扩张，Service Account（SA）权限过度授予（Over-privileged）的问题愈发普遍。许多组织的 SA 拥有远超实际需要的 RBAC 权限，这是 Kubernetes 安全事件中的高频风险点 。然而，手动审计所有命名空间下的 Role、ClusterRole 及其绑定关系是一项极其繁琐且容易遗漏的工作。

三、竞品分析

目前国际开源社区中已涌现出若干将 AI 引入 Kubernetes 运维的工具，但整体上呈现"能力单点化、整合度低"的特征，与 KubeWise 的设计目标存在显著差距。

3.1 主要竞品概览

K8sGPT 是当前最具代表性的 Kubernetes AI 运维工具，作为 CNCF Sandbox 项目受到广泛关注 。其核心能力是扫描集群资源（Pod、Service、Deployment 等），并将原始的 Kubernetes 错误事件翻译成人类可读的故障解释，支持 OpenAI、Azure OpenAI、Cohere、Amazon Bedrock、Gemini 等多种模型后端，并提供 Operator 模式和 MCP Server 支持。然而，K8sGPT 存在明显的功能边界：**它以集群状态分析为起点，而非端到端的运维操作**，不支持"部署一个应用"这类主动操作，缺乏跨资源的联合推理查询能力，也不具备 RBAC 安全审计功能。

kubectl-ai（Google 开源）是一款自然语言转 kubectl 命令的 CLI 工具，能将用户意图映射为相应的 Kubernetes 命令并执行。但它本质上是一个命令生成器，缺乏多步骤规划与推理能力，无法处理需要"先查询、再分析、再决策"的复杂任务。

HolmesGPT（Robusta Labs 开源）支持接入 Prometheus、Grafana、Loki、GitHub 等多种数据源，面向持续调查型工作流，具有更广泛的数据覆盖面 。但其配置复杂，需要依赖完整的可观测性基础设施，不适合轻量化或学习场景；同时同样不支持主动部署操作与安全审计。

Metoro、**Komodor Klaudia** 等商业平台提供了更完善的生产级 AI SRE 能力，但均为付费商业产品，不具备开源可定制性，且与国内主流 LLM（DeepSeek、通义千问等）的集成存在障碍。

3.2 竞品局限性总结

综合分析可以看出，现有工具普遍存在**"分析能力与操作能力割裂"**、**"单任务执行与多步推理失衡"**以及**"安全治理能力缺失"**三大共性局限。K8sGPT 擅长解释问题但无法主动干预；kubectl-ai 能执行命令但无法深度推理；HolmesGPT 具备调查能力但依赖重型基础设施；商业方案功能完善但封闭且成本高昂。**KubeWise 的核心价值在于将上述分散能力统一整合为一个端到端的、开源的、对话驱动的运维智能体**，并通过 ADK 有状态多步推理架构填补现有工具在复杂任务规划上的空白。

四、实现方案

4.1 系统总体架构

KubeWise 采用**多 Agent 协作架构**，以 ADK 构建的有状态执行图为核心，将自然语言理解、任务规划、工具调用与结果汇报整合为一条完整的闭环流水线。

4.2 核心模块设计

4.2.1 意图理解与路由模块

系统接收到用户的自然语言输入后，首先由 **Router Agent** 对输入进行意图分类，区分"操作类"、"查询类"、"故障排查类"和"安全审计类"四大任务类型，并提取关键实体（如目标命名空间、资源名称、应用名称等）。Router Agent 基于 LLM 的 Function Calling 能力实现，通过 System Prompt 约束输出格式为结构化 JSON，确保下游 Agent 的调度准确性。

对于涉及多个能力域的复合请求（如"帮我检查 production 命名空间中有没有崩溃的 Pod，如果有，分析日志并告诉我原因"），Router 会将任务分解为顺序执行的子任务链，通过 ADK 的状态图节点传递上下文。

4.2.2 操作 Agent（一句话操作）

操作 Agent 负责将自然语言部署或管理指令转化为具体的 Kubernetes 操作序列。以"帮我部署一个 ArgoCD"为例，Agent 的推理链如下：

1. **规划阶段**：确认目标命名空间（默认 `argocd`），检查是否已存在同名 Namespace；
2. **工具调用**：通过 Helm SDK 拉取 `argo/argo-cd` Chart，生成 values 配置；
3. **执行阶段**：调用 Kubernetes Golang Client 创建 Namespace，执行 Helm install；
4. **验证阶段**：轮询 ArgoCD Deployment 的 Ready 状态，确认部署成功；
5. **汇报阶段**：向用户返回访问地址、初始密码获取方式等关键信息。

整个流程由 ADK 的有状态图驱动，每个节点均配置有回退策略（如 Namespace 已存在时跳过创建步骤），确保幂等性与可靠性。

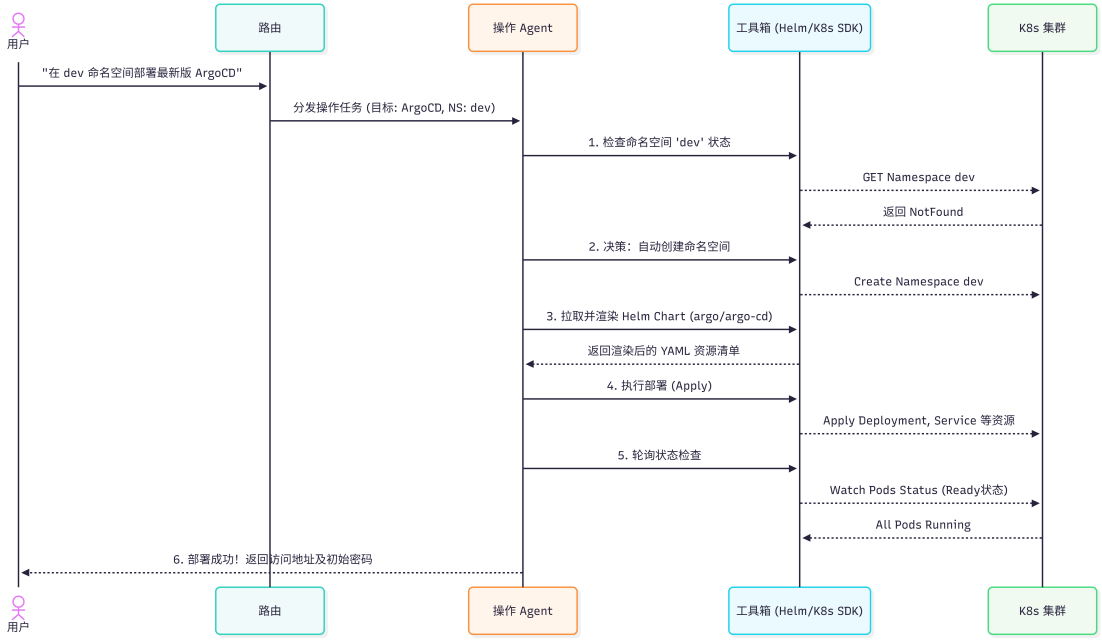


图4.1：操作Agent示意流程图

4.2.3 查询 Agent（智能查询）

查询 Agent 通过多轮工具调用，实现跨资源的联合推理。以"哪个 PV 占用的空间最大，挂载到了哪个 Pod"为例：

1. 调用 `list_persistent_volumes()` 工具获取全量 PV 列表及其容量信息；
2. 按容量排序，确定最大的 PV 及其绑定的 PVC 名称（`claimRef`）；
3. 遍历所有 Pod 的 Volume 配置，找到引用该 PVC 的 Pod；
4. 汇总结果，以结构化自然语言返回给用户。

该模块的设计要点在于**工具的原子化与组合性**：每个工具函数对应一个单一的 K8s API 操作，查询 Agent 通过 LLM 的推理能力将多个原子工具组合为解决复杂问题的完整路径，无需预定义固定查询模板，具有极强的通用性与可扩展性。

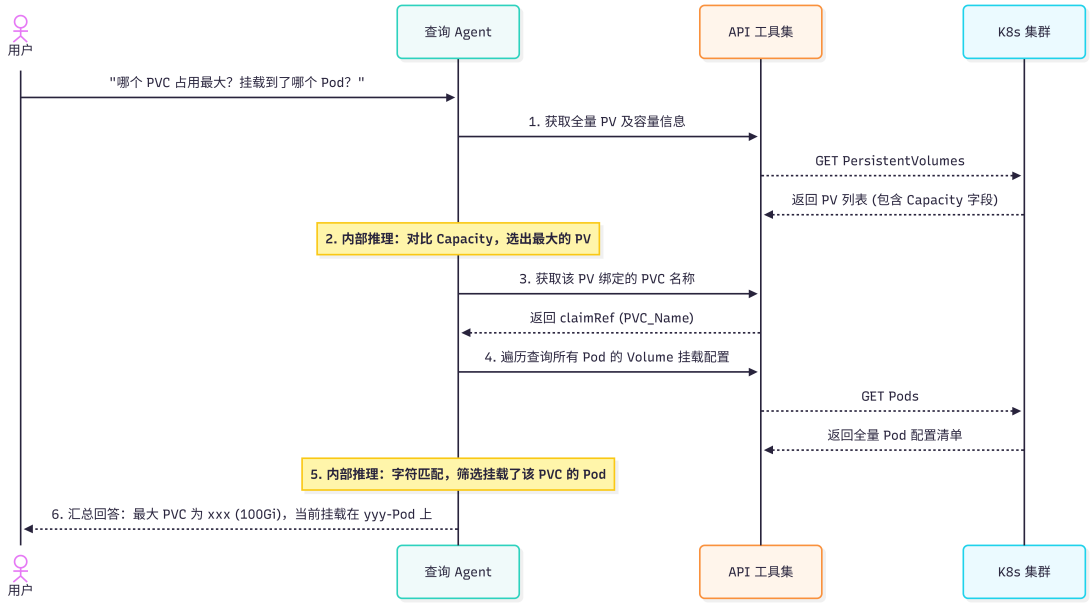


图4.2：查询Agent示意流程图

4.2.4 故障排查 Agent（自动故障排查）

故障排查 Agent 在检测到异常信号（容器 Unhealthy、CrashLoopBackOff、OOMKilled 等）后，自主启动诊断流程：

1. **异常检测**：主动监听 K8s Events，通过 Watch API 实时捕获 Warning 类型事件；
2. **上下文收集**：自动拉取异常 Pod 的近期日志（`kubectl logs --previous`）、Describe 信息及相关 Deployment/ConfigMap 配置；
3. **智能分析**：将收集到的上下文构建为结构化 Prompt，交由 LLM 进行根因分析，输出"问题摘要 → 疑似根因 → 建议修复步骤"三段式诊断报告；
4. **可选修复**：经用户确认后，Agent 可自动执行建议的修复操作（如调整资源限制、重启 Deployment、修正环境变量等）。

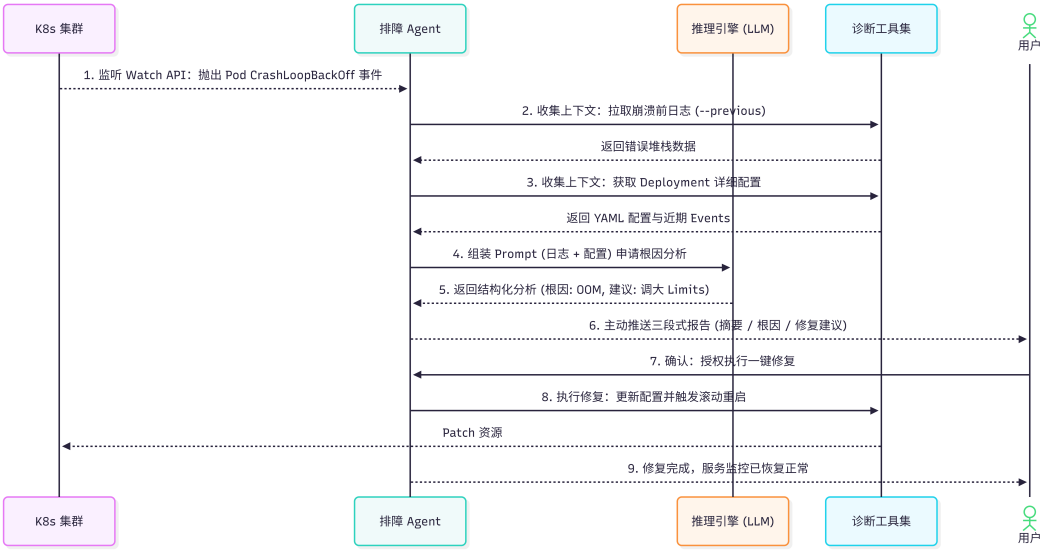


图4.3：故障排查Agent示意流程图

4.2.5 安全审计 Agent（安全检测）

安全审计 Agent 定期或按需对集群的 RBAC 配置进行扫描，识别权限过度授予的 Service Account。核心审计规则包括：

- **通配符权限检测**：识别 `verbs: ["*"]` 或 `resources: ["*"]` 的高危规则；
- **跨命名空间提权检测**：识别绑定了 ClusterRole 的命名空间级 ServiceAccount；
- **特权动词检测**：识别拥有 `exec`、`delete`、`escalate`、`impersonate` 等高危动词的 Role；
- **默认 SA 过权检测**：识别 `default` ServiceAccount 被绑定非空 Role 的情况。

审计结果按风险等级（高/中/低）分类，并由 LLM 生成自然语言格式的审计报告与加固建议，支持导出为 Markdown 格式的合规报告。

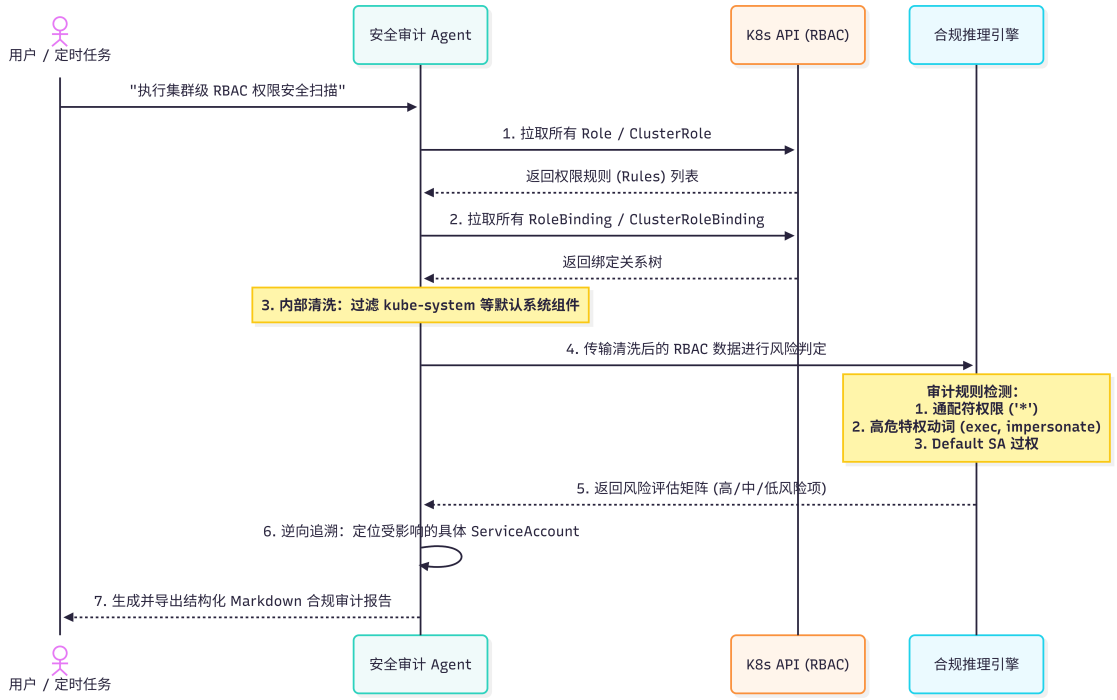


图4.4：安全审计Agent示意流程图

4.3 关键技术决策

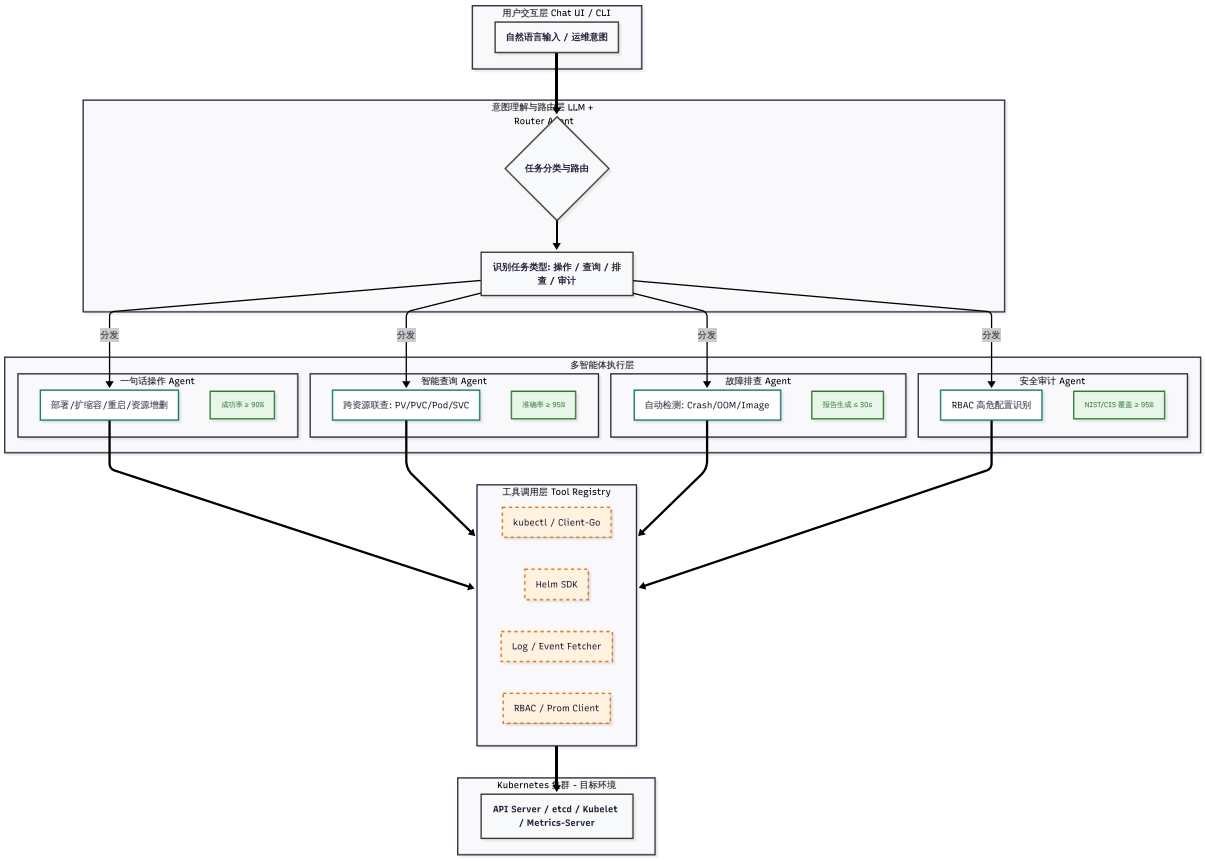
为什么选择 ADK-Go 而非直接使用 LangChain？

LangChain 的传统 Agent 模式（AgentExecutor）基于线性链式结构，在处理需要条件分支、循环重试或多 Agent 协作的复杂工作流时，扩展性受限。相比之下，ADK-Go 提供了更适合复杂编排的底层架构，不仅通过状态共享和节点路由天然支持多步骤规划、错误回退和人工审批（Human-in-the-Loop）；更重要的是，作为 Go 原生框架，ADK-Go 能够无缝对接 Kubernetes 原生的 `client-go` 生态。这对于 K8s 运维这类"先查询状态、再决策操作"的有状态任务场景，不仅在工作流架构上更契合，在并发性能、内存占用和云原生部署上也具备压倒性优势。

为什么支持 MCP 协议集成？

Model Context Protocol（MCP）是 Anthropic 于 2024 年 11 月提出的工具调用标准协议，正在成为 AI 工具集成领域的事实规范 。K8sGPT 等工具已提供 MCP Server 支持，KubeWise 通过将核心工具封装为 MCP Server，一方面与主流 AI 助手（Claude Desktop、Cursor 等）形成原生集成，另一方面保留了独立 CLI / Web UI 的交互入口，构建了面向未来的开放工具生态。

4.4 架构逻辑



五、技术栈

本项目技术栈的选型遵循**开源优先、生产可用、国内可达**三大原则，确保项目在国内高校实验环境和企业私有云环境中均可顺利部署。

5.1 技术栈总览

层次	技术选型	版本 / 说明
Agent 框架	google/adk-go	v1.1.0+ / 有状态多 Agent 编排
LLM 后端	GLM-5.1 / Qwen3.6	支持多模型切换，优先国产模型
Kubernetes 接入	kubernetes Golaog Client	官方 SDK，支持全量 K8s API
应用部署	helm.sh/helm	v4 / Helm Chart 管理
工具协议	MCP（Model Context Protocol）	标准工具接口，支持外部 AI 客户端调用
可观测性集成	K8s Events Watch API	指标与事件监听
安全审计	自研 RBAC Analyzer + Kubescape（可选）	ClusterRole/RoleBinding 分析

层次	技术选型	版本 / 说明
后端服务	Echo + WebSocket	Agent 服务 API 层
前端界面	React + TailwindCSS	对话式 Web UI
容器化部署	Docker + Helm Chart（自部署）	支持以 Operator 形式在集群内运行
测试环境	kind / minikube	本地 K8s 集群，快速验证

5.2 LLM 选型说明

KubeWise 设计为**模型无关（Model-Agnostic）**的架构，通过 ADK 的统一 `NewModel` 接口抽象底层 LLM 调用。在优先级上，推荐使用以下模型：

- **DeepSeek-V3 / DeepSeek-R1**（首选）：国产开源模型，推理能力强，API 价格低廉，适合工具调用场景 [10]；
- **通义千问 Qwen3.5-72B**：阿里开源，中文理解能力优秀；
- **OpenAI GPT-4o / GPT-4.1**：作为基准对照和高精度场景的备选。

所有模型均通过统一的 `config.yaml` 配置切换，无需修改业务代码。

六、预期效果

6.1 功能验收标准

项目计划在初赛与选拔赛阶段达到以下功能完整度：

功能模块	预期能力描述	验收指标
一句话操作	支持 10+ 种常用运维操作（部署 Helm 应用、扩缩容、重启 Pod、创建/删除资源等）	自然语言转操作成功率 ≥ 90%（标准测试集）
智能查询	支持 5+ 类跨资源联合查询（PV/PVC/Pod 关联、Service/Endpoint 关系、资源用量排行等）	查询结果准确率 ≥ 95%
自动故障排查	自动检测 CrashLoopBackOff、OOMKilled、ImagePullBackOff 等常见故障模式	从告警到生成诊断报告 ≤ 30 秒
安全审计	识别高危 RBAC 配置（通配符权限、特权动词、默认 SA 过权）	覆盖率 ≥ 95%（基于 NIST/CIS Kubernetes Benchmark）

6.2 演示场景规划

以下四个场景将作为项目演示的核心用例，分别体现系统的四大核心能力：

场景一——一句话部署（操作能力）
> 用户输入："帮我在 `dev` 命名空间里部署最新版的 ArgoCD，并暴露 NodePort"
>
> Agent 执行流程：检查命名空间 → 添加 Helm Repo → 生成 values.yaml → 执行 Helm install → 修改 Service 类型 → 返回访问地址与初始密码

场景二——跨资源智能查询（查询能力）
> 用户输入："现在哪个 PVC 占用空间最大？它挂载到了哪个 Pod 上，Pod 所在节点的磁盘还剩多少？"
>
> Agent 执行流程：列举 PVC → 排序取最大 → 反查绑定 PV → 定位挂载 Pod → 查询 Pod 节点 → 拉取节点磁盘指标 → 汇总回答

场景三——自动故障排查（排障能力）
> 触发条件：集群中某 Deployment 的 Pod 进入 CrashLoopBackOff 状态
>
> Agent 执行流程：Watch Event → 捕获 BackOff 事件 → 拉取 `--previous` 日志 → 构建诊断 Prompt → LLM 分析（识别 OOM / 配置错误 / 依赖缺失等根因）→ 输出三段式诊断报告 → 可选自动修复

场景四——RBAC 安全审计（安全能力）
> 用户输入："帮我检查一下集群里有没有权限过大的 Service Account"
>
> Agent 执行流程：枚举所有 ClusterRole/Role → 识别通配符与特权动词 → 关联 RoleBinding/ClusterRoleBinding → 追溯绑定的 SA → 按风险等级分类 → 生成 Markdown 审计报告

6.3 应用价值展望

运维效率提升：根据对类似场景的分析，通过 AI Agent 辅助，重复性运维操作的平均处理时间可从分钟级降低至秒级；故障诊断的 MTTR 有望缩短 60% 以上。

知识民主化：KubeWise 将深度的 Kubernetes 专家知识封装为对话接口，使得没有 K8s 专项经验的开发者也能安全地完成日常运维操作，显著降低云原生技术的应用门槛。

安全治理价值：面对日益严峻的云原生安全形势，KubeWise 的自动 RBAC 审计功能可帮助组织持续维护最小权限原则，弥补人工审计的覆盖盲区。

产业化潜力：随着国内企业 Kubernetes 集群规模的持续增长，智能运维平台具有明确的商业化路径，可通过 SaaS 订阅、私有化部署或与现有 DevOps 平台集成等多种模式落地。